

Front-End UI Stack Bake-Off Kit

A structured way to trial React, Angular, Vue, Next.js and Blazor for replacing legacy UI — everyone builds the **same** small CRUD app, then we score it together. For discussion and evaluation, not a decision on tech.

Common feature: Parts CRUD (form + table) **Format:** Time-boxed bake-off **Date:** August 2026

1 Why build the same thing five times

The only fair way to compare stacks is to hold the problem constant and let the differences surface.

Vendor benchmarks and "X vs Y in 2026" blog posts measure toy cases and rarely reflect *your* team, *your* patterns, or the legacy screens you actually need to replace. A bake-off fixes that: pick one representative screen, give every framework the identical spec and acceptance criteria, and have a dev (or pair) implement it in a fixed time-box. What you're really measuring isn't "which framework is fastest" — it's **how it feels to build, read, test and maintain a typical PartsCheck screen**, plus a few objective numbers on top.

Keep it honest: same data model, same validation rules, same mock API, same styling ambition. Differences in effort and code then mean something.

2 The common app — "Parts Manager"

A single-screen CRUD on a **Part** entity: a table of parts plus a form to create/edit. Small enough to build in an afternoon, real enough to exercise the things that matter.

Data model — **Part**

FIELD	TYPE	RULES
id	string/guid	server-assigned
sku	string	required, unique, 3–20 chars
name	string	required, ≤ 80 chars
category	enum	required (dropdown)
price	decimal	required, > 0
stock	integer	required, ≥ 0
active	boolean	toggle, default true
updatedAt	datetime	server-set

Screen behaviour

- ☐ **Table:** paginated (10/page), sortable columns, text filter on SKU/name, category filter
- ☐ **Row actions:** Edit and Delete (Delete asks to confirm)
- ☐ **Form:** Add/Edit in a modal or side panel, inline field validation, disabled Save until valid
- ☐ **Feedback:** loading, empty, and error states; success toast on save/delete
- ☐ **Persistence:** talks to a shared mock REST API (below), not just in-memory

Required layout & component architecture

The screen must be assembled as a proper **app shell built from separate, self-contained components** — not one monolithic file. This is where a framework's component model, composition and reuse story really shows.

☐ **Header** — its own component (brand, page title, a user/menu slot)

☐ **Sidebar menu** — its own component (navigation items, collapsible)

☐ **Content area** — its own component, hosting the Parts Manager

Header, sidebar and content must be three distinct components composed by a parent layout — pass data/callbacks across boundaries, don't inline everything.

Must exercise these controls (build or use a library — note which):

☐ **Typeahead / autocomplete** — e.g. part or supplier search with async suggestions

☐ **Data grid** — the parts table with sort, filter and pagination

☐ **Modal dialog** — the add/edit form and the delete confirmation

Shared mock API contract

So every implementation does the same network work, mock one REST resource. Fastest option is `json-server` (one command, zero code) — everyone points at the same endpoints:

```
# one shared backend for all four apps
npx json-server --watch parts.json --port 4000

# endpoints every framework consumes identically
GET    /parts?_page=1&_limit=10&_sort=name&q=filter
POST   /parts
PUT    /parts/:id
DELETE /parts/:id
```

Blazor Server/WebAssembly hits the same URL via `HttpClient`; the JS stacks via `fetch`. That keeps the data layer neutral so you're comparing the *UI* stack, not the backend.

3 Acceptance criteria (identical for all five)

A build "counts" only when every box is ticked. Hand this checklist to each dev up front.

- ☐ Page is an app shell of **three separate components** — Header, Sidebar menu, Content — composed by a parent layout
- ☐ Sidebar navigation works and the layout is reused, not duplicated per screen
- ☐ A **typeahead/autocomplete** control is used (e.g. part/supplier search with suggestions)
- ☐ The parts table is a **data grid** with sort, filter and pagination
- ☐ Add/Edit and Delete-confirm are presented in a **modal dialog**
- ☐ Table loads parts from the mock API with a visible loading state
- ☐ Pagination, column sort, and SKU/name text filter all work against the API
- ☐ Category dropdown filters the table
- ☐ "Add part" opens a form; all validation rules enforced with inline messages
- ☐ Save is disabled while the form is invalid or submitting
- ☐ Editing an existing row pre-fills the form and updates via **PUT**
- ☐ Delete prompts for confirmation, then removes via **DELETE**
- ☐ Empty state (no parts) and error state (API down) both render gracefully
- ☐ Success feedback (toast/banner) after create, update, delete
- ☐ Basic responsive layout; no console errors; passes a production build
- ☐ At least one component/unit test demonstrating the framework's test story

Keep styling deliberately modest — a plain table and form. If someone reaches for a component library (MUI, Vuetify, MudBlazor, shadcn), note it: "batteries included" vs "bring your own" is itself a data point worth capturing.

4 The contestants

One-line orientation, a starter command, and honest watch-outs for each. Assign one dev or pair per stack.

■ React (SPA) ■ Angular ■ Next.js ■ Vue ■ Blazor

LIBRARY · CLIENT-RENDERED SPA

React + Vite

The baseline. Huge ecosystem, deepest hiring pool, most "how do I..." answers. You assemble routing/data-fetching yourself.

```
npm create vite@latest \
  parts-react -- --template react-ts
```

- **shines** flexibility, talent availability, TanStack Query/Table
- **watch** decision fatigue — many ways to do everything

FRAMEWORK · FULL, OPINIONATED

Angular

The batteries-fully-included option: routing, forms, HTTP, DI and testing all first-party, TypeScript-first. Strong fit for large, long-lived line-of-business apps and .NET-style teams who like structure and convention.

```
npm i -g @angular/cli
ng new parts-angular
```

- **shines** one blessed way to do things, typed forms, enterprise scale
- **watch** steeper ramp, more boilerplate/ceremony than Vue or React

META-FRAMEWORK · REACT + SSR

Next.js

React with routing, server rendering and server actions built in. Great if screens need SEO/SSR or a full-stack story; heavier than a plain SPA for an internal tool.

```
npx create-next-app@latest \
  parts-next
```

- **shines** full-stack, App Router, data-fetching conventions
- **watch** server/client component model has a learning curve; overkill for pure internal CRUD

FRAMEWORK · PROGRESSIVE

Vue 3

Gentle learning curve, excellent single-file components, batteries closer to included (Router, Pinia). Often the fastest to a clean CRUD screen.

```
npm create vue@latest
# (or --template vue-ts via Vite)
```

- **shines** readability, quick ramp, official router/state
- **watch** smaller hiring pool locally than React

Blazor (.NET 9/10)

Build the UI in C#, share models with your backend, one language end-to-end. The unified "Blazor Web App" now mixes Server, WebAssembly and static render modes per component.

```
dotnet new blazor \
  -o PartsBlazor --interactivity Auto
```

- **shines** C# reuse, strong typing, one .NET skillset
- **watch** pick render mode deliberately (Server vs WASM vs Auto); smaller JS-interop ecosystem

If most PartsCheck legacy is .NET, weight Blazor's "same language/models as the backend" advantage heavily — it can dwarf raw framework benchmarks when it comes to long-term maintenance cost and who on the team can own the code.

5 What to measure

A mix of a few hard numbers and honest human judgement. Record both.

Objective (capture these numbers)

- ☐ **Time to build** — wall-clock hours to pass all acceptance criteria
- ☐ **Lines of code** — `cloc src` on the app folder
- ☐ **Production bundle** — output size after a prod build (KB gzipped)
- ☐ **Cold load** — Lighthouse/DevTools first-load time on a throttled profile
- ☐ **Dependencies** — count of direct deps added beyond the starter
- ☐ **Build/CI time** — clean install + build duration

Subjective (each dev writes 3–5 sentences)

- ☐ **Developer experience** — hot reload, tooling, error messages
- ☐ **Readability** — would a teammate understand it cold?
- ☐ **Validation/forms story** — how painful was the form?
- ☐ **Testing story** — how easy was the one test?
- ☐ **Docs & AI assist** — quality of docs and copilot answers
- ☐ **"Would I maintain 50 of these?"** — the gut-check

6 Scoring rubric

Weighted so the things that actually drive a legacy-replacement decision count most. Score each framework 1–5 per row; the sheet in §7 does the weighting.

DIMENSION	WEIGHT	WHAT A 5 LOOKS LIKE
Fit with existing skills/stack	20%	Team already fluent; reuses current backend/models
Developer experience & speed	18%	Fast to build, great tooling, clear errors
Maintainability & readability	18%	New hire reads it cold; low ceremony
Ecosystem & component libraries	12%	Mature tables/forms/UI kits available
Hiring & talent availability	12%	Easy to hire/contract for locally
Performance & bundle size	8%	Small bundle, snappy load & interaction
Testing & tooling maturity	7%	First-class test runner, easy component tests
Longevity & governance	5%	Backed, active releases, low churn risk
Total	100%	

Weights are a starting point — adjust them together *before* anyone builds, so the scoring reflects PartsCheck's real priorities rather than being reverse-engineered to fit a favourite.

7 Score sheet

Fill after the bake-off. Enter 1–5 per cell; weighted total = $\Sigma(\text{score} \times \text{weight})$.

DIMENSION	WT	REACT	NEXT.JS	VUE	BLAZOR	ANGULAR
Fit with existing skills/stack	20	—	—	—	—	—
Developer experience & speed	18	—	—	—	—	—
Maintainability & readability	18	—	—	—	—	—
Ecosystem & libraries	12	—	—	—	—	—
Hiring & talent	12	—	—	—	—	—
Performance & bundle	8	—	—	—	—	—
Testing & tooling	7	—	—	—	—	—
Longevity & governance	5	—	—	—	—	—
Weighted total (/5)		—	—	—	—	—

Objective numbers

METRIC	REACT	NEXT.JS	VUE	BLAZOR	ANGULAR
Time to build (hrs)	—	—	—	—	—
Lines of code	—	—	—	—	—
Prod bundle (KB gz)	—	—	—	—	—
Cold load (s)	—	—	—	—	—
Direct deps added	—	—	—	—	—

8 Don't reinvent it — proven existing projects

Before (or instead of) building from scratch, these open-source projects already implement the same app across many frameworks. Great for reading real code side-by-side and for stealing a spec.



RealWorld / "Conduit" — the mother of all demo apps

A full Medium-style app (auth, CRUD articles, comments, profiles) implemented against one shared API spec in **dozens** of front-ends including React, Next.js, Vue and ASP.NET/Blazor. This is the closest thing to your exact ask — same real-world app, many stacks, browsable side-by-side. Live demos at codebase.show.



TodoMVC — the same to-do app in every framework

Smaller scope than a CRUD table, but the canonical "how does the same UI look in React vs Vue vs ..." reference. Good for a quick feel of syntax and structure before committing to a full bake-off.



Component Party — side-by-side syntax comparison

Shows the same concept (state, props, loops, forms, lifecycle) in React, Vue, Svelte, Solid and more, snippet-by-snippet. The fastest way to eyeball "which of these reads best to our team".



js-framework-benchmark (Krausest) — the reference perf benchmark

Independent, continuously-run benchmark of table/DOM operations across virtually every JS framework. Use it for the *performance* row rather than trusting blog numbers — but remember it measures raw DOM churn, not real-app maintainability.



RealWorld — .NET / Blazor implementations

The ASP.NET Core / Blazor RealWorld builds, useful if you want to see the C# end-to-end story on the same shared spec as the JS versions.

Recommended combo: skim **Component Party** for a first feel, read the **RealWorld** implementations of your shortlisted stacks for real code at scale, then run **your own Parts Manager bake-off** for numbers and opinions grounded in PartsCheck's actual context.

9 Suggested run of play

- ☐ **Day 0 — Agree the rules.** Confirm the spec, lock the weights in §6, assign one dev/pair per framework, stand up the shared `json-server`.
- ☐ **Build (½–1 day each).** Everyone builds Parts Manager to the acceptance criteria, time-boxed. No peeking at each other's code until done.
- ☐ **Capture.** Each dev records objective numbers (§5) and writes their 3–5 sentence subjective notes while it's fresh.
- ☐ **Show & tell.** 15 min demo per stack — same screen, five times. The team sees the differences live.
- ☐ **Score together.** Fill the sheet in §7 as a group; discuss disagreements rather than averaging them away.
- ☐ **Decide the frame.** Output isn't necessarily "the winner" — it's a shared, evidence-based view to take into the legacy-replacement discussion.

PartsCheck Front-End UI Stack Bake-Off Kit · prepared for discussion and evaluation · August 2026.

Framework starter commands reflect current tooling (Vite, create-next-app, create-vue, .NET Blazor Web App); verify versions when you scaffold. Perf figures should come from your own runs or the linked live benchmark, not this document.